Northeastern University, Khoury College of Computer Science

CS 6120 Natural Language Processing
DreamRAG: Visualize the compositions of your unique dreams

Gin Fu, Shiran Wang
fu.jingyi2@northeastern.edu, wang.shira@northeastern.edu
github.com/blueif16/dreamrag

1 Proposed Objective

DreamRAG helps people make sense of their own dream journals. Each query returns a bento-grid dashboard instead of a block of text, composed on the fly from three corpora: community narratives, dream-analysis literature, and the user's own journal. A self-correcting LangGraph agent drives the retrieval. We fuse BM25 and pgvector cosine scores using Reciprocal Rank Fusion, or RRF (Cormack et al., 2009), then expand the seed set by walking typed edges in a Supabase `doc_relations` table through a recursive CTE. The pipeline routes results to about a dozen widgets: interpretation synthesis, symbol graph, emotion radar, community mirror, heatmap, stat card. Each consumes a different slice of the retrieval output. Raw chunks, SQL aggregates, or typed edges, depending on the widget. And each carries chunk-level provenance back to its source. So retrieval decides both what appears and how it gets rendered.

2 Background and Motivation

Problem. People who keep dream journals end up with years of rich longitudinal text. What they do not have is a scalable way to connect those entries to the frameworks that would make sense of them: Jung on symbols, Freud on latent content, the Hall/Van de Castle (HVdC) coding system, or Domhoff's continuity-hypothesis account. The same gap exists at the community level, where public datasets hold tens of thousands of dreams that nobody has cross-indexed against private archives. Current practice is manual symbol lookup and cross-referencing by hand. It does not scale. If it works, journalers get a reflective tool tied to the interpretive tradition, researchers can cross-reference private and public dream data at scale, and the pattern works in other domains where flat top-k is not enough.

Why it is hard. Single-pass RAG returns a flat top-k list. That is not enough here. Dream analysis wants multi-hop symbolic reasoning, cross-corpus triangulation, and temporal co-occurrence discovery inside one user's archive. It also wants provenance, because mixing clinical literature with personal journal data without traceability is a liability. The corpora themselves look nothing alike: informal first-person journals, terse HVdC codes, academic prose. And the matching is symbolic, not lexical. "Water" should reach Jung's "unconscious" through a typed edge, not through word overlap.

Related literature. The interpretive backbone comes from the Hall/Van de Castle manual (dreams.ucsc.edu/Coding), Jung, Freud, and Domhoff (MIT Press, 2018). For NLP-scale precedent we look to *Our Dreams, Our Selves* (2020) and the Reddit `r/Dreams` topic-model study (Sanz et al., 2025). Technical foundations are RRF (Cormack et al., 2009), HNSW (Malkov & Yashunin, 2018), and Qwen3-Embedding (Zhang et al., 2025), whose instruction-prefixed query format we adopt.

Data. We only ingest community sources that come pre-annotated, so HVdC codes come from the source rather than from an LLM at ingest time. Table 2.1 lists the two ingested community sources (∼48K annotated narratives), the textbook layer, and the per-user Supabase entries. Unannotated variants like DreamBank/DReAMy-lib, SDDb, and Reddit `r/Dreams` are on hold until we have a runtime auto-tagger.

Source	Scale	Annotations / notes
DreamBank Annotated	∼28K reports	HVdC emotion + character codes (HF `gustavecortal/DreamBank-annotated`)
Dryad HVdC-annotated	∼20K reports	Full HVdC codes + numeric indices (Dryad 10.5061/dryad.qbzkh18fr)
Freud, Jung (Gutenberg)	textbook	Chunked at 1500/200
HVdC coding manual	reference	Scraped from dreams.ucsc.edu/Coding

Table 2.1: Ingested sources. Per-user Supabase entries are populated at runtime and auto-tagged only when source annotations are absent.

3 Proposed Approach and Implementation

Pre-processing. Corpora come in through the `RAGStore` and `DataAdapter` components of the Supabase RAG scaffold. Academic texts chunk at 1500/200 tokens, the scraped HVdC manual at 1200/150. Dream narratives stay intact at report level. We parse HVdC codes and numeric indices directly from source CSV and TSV columns into `metadata`, so bulk ingest never calls an LLM. User-written journal entries, which ship with no source annotations, go through a planned 4B-class Qwen tagger. Edges come out of the same pass. In-archive symbol co-occurrences accumulate weight over repetition, and cross-corpus `similar_to` edges link personal entries to community and academic chunks.

Retrieval algorithm. Retrieval runs entirely in Supabase Postgres through the scaffold's *Context Mesh*. RRF fuses BM25 and pgvector cosine scores over `vector(1024)` HNSW embeddings into a seed set. A recursive CTE then walks that seed set outward along seven typed edges in `doc_relations`: `symbolizes`, `similar_to`, `co_occurs`, `follows`, `interprets`, `contradicts`, `coded_as`. The whole thing sits inside a self-correcting LangGraph: `classify_query` → `plan_retrieval` → `retrieve` → `grade` → `rewrite_retry` → `synthesize`.

Widget composition. The `synthesize` node emits layout JSON that maps retrieval results onto roughly fourteen widget types, each taking a different slice. Raw chunks drive the *Interpretation Synthesis* and *Textbook Card*. Typed edges drive the *Symbol Network* (`co_occurs`), *Dream Timeline* (`follows`), and *Community Mirror* (cross-corpus `similar_to`). SQL aggregates over `user_dreams` drive the *Heatmap*, *Emotion Radar*, and *Stat Card*. Composition rules keyed on query type pick which widgets appear and at what grid size, so the same query vocabulary yields structurally different dashboards depending on what was asked.

Robustness. The `grade` node runs an LLM relevance check that triggers a query rewrite on failure, using a different model family than the synthesizer to avoid self-preference bias. The three namespaces (`community_dreams`, `dream_knowledge`, `user_{uid}_dreams`) stay isolated at query time and each receives instruction-prefixed queries. Class imbalance is stark. Roughly 48K community reports against a much smaller textbook layer and per-user archives in the tens to hundreds. We handle it with stratified per-namespace budgets and HVdC frequency capping. Graph over-fitting is controlled by normalizing `co_occurs` weights by user entry count and validating on a held-out user split.

Inference stack. Everything runs locally on a GCP `g2-standard-8` with an NVIDIA L4 (24 GB VRAM). No cloud APIs in the serving path. Chat uses `unsloth/Qwen3.6-35B-A3B-GGUF:UD-IQ4`, a MoE that activates about 3B parameters per token, so throughput tracks a dense 3B rather than a 35B. The freed VRAM lets us keep the KV cache at fp16 for 32K context, which matters because q8_0 KV degrades long-context needle retrieval and tool-call argument fidelity. Both are load-bearing for the graph-hop RAG and the agent loop. A dense Qwen3.5-27B at Q4 stays available as a fallback. Embeddings use Qwen3-Embedding-0.6B (Q8_0, 1024-dim MRL, last-token pooling, L2-normalized), with the option to drop the query-time dimension to 768 or 512 if latency binds. HNSW on `vector(1024)` and GIN indexes on symbol and emotion tag arrays handle hybrid filtering. Namespace-scoped queries keep the recursive CTE bounded.

Validation. Evaluation answers four questions. Does retrieval find the right dreams? We compare the graph-expanded pipeline against BM25, pgvector, and plain RRF on 100 hand-labeled similar-dream pairs from DreamBank, which isolates what the graph layer adds. Do the interpretations hold up? Three raters score synthesized cards on a held-out slice of Dryad dreams the graph never saw during training. Is the provenance honest? We audit twenty dashboards by hand, checking that each widget's cited chunks actually support what it rendered. And does any of this help real journalers? We also track how much of a typical answer comes from graph hops rather than the first-pass seed set, which tells us whether the graph is doing real work. Operating targets are under two seconds to first token and under fifty milliseconds per embedding.

Limitations. Provenance guarantees retrieval traceability, not generator faithfulness, so a post-synthesis faithfulness check runs against the linked chunks. Cold-start users get seed-only layouts until their archive grows. Low-bit MoE quantization may degrade long-chain reasoning, which we probe by comparing UD-IQ4_XS against a higher-bit reference. Every corpus is English and Western-theory-dominated, which limits cross-cultural generalization. DreamRAG is a research prototype, not a clinical tool. Every interpretation card carries a disclaimer to that effect.